

CMT 451 Module 2

CMT 451 Module 2: Mobile OS Security

(10-Slide Visual Summary)

Slide 1: Module 2 Title & Intro

CMT 451 Mobile OS Security

The Fortress in Your Pocket – Android vs. iOS

Instructor: Dr. Stanley Mwangi Chege
[University Name]

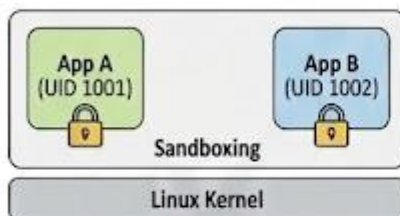


Slide 2: Learning Outcomes

Objectives

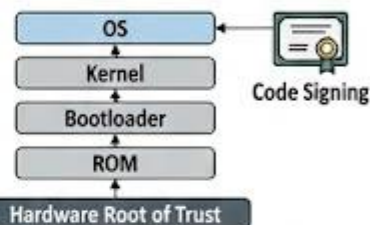
- ✓ Evaluate Android vs. iOS architectures.
- ✓ Analyze OS-level controls (Sandboxing, Permissions).
- ✓ Apply ISO/IEC 27001:2022 controls.

Slide 3: Android Security Model (The Linux Core)



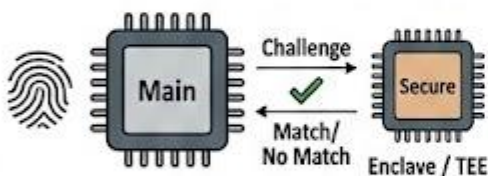
Each app is a unique user. Kernel prevents access.

Slide 4: iOS Security Model (The Walled Garden)



Secure Boot Chain and Code Signing.

Slide 5: Biometrics & The Secure Enclave



Biometric data is isolated in secure hardware.

Slide 6: Standard Mapping – NIST SP 800-53

NIST



AC-3: Access Enforcement



AC-19: Mobile Access Control

NIST controls for access & device security.

Slide 7: Standard Mapping – ISO/IEC 27001:2022



Slide 8: Data Protection – Encryption



FBE allows granular security and Direct Boot.

Slide 9: Network Security – Data in Transit



Android: Network Security Config
iOS: App Transport Security (ATS)

Enforces secure connections to prevent MitM.

Slide 10: Summary & The Rooting/Jailbreaking Risk



Jailbreaking = Superuser Access
Breaks Sandboxing
Violates ISO 27001 Integrity

Compromising OS integrity for control risks security.

Infographic for CMT 451 Module 2: Mobile OS Security (10-Slide Visual Summary)

Slide 1: Module 2 Title & Intro

CMT 451 Mobile OS Security

The Fortress in Your Pocket –
Android vs. iOS

Instructor: Dr. Stanley Mwangi Chege
[University Name]

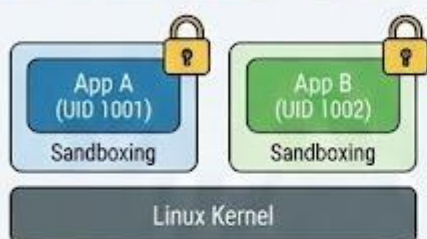


Slide 2: Learning Outcomes

Objectives

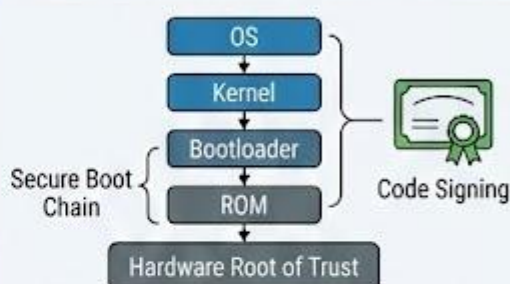
- ✓ Evaluate Android vs. iOS architectures.
- ✓ Analyze OS-level controls (Sandboxing, Permissions).
- ✓ Apply ISO/IEC 27001:2022 controls.

Slide 3: Android Security Model (The Linux Core)

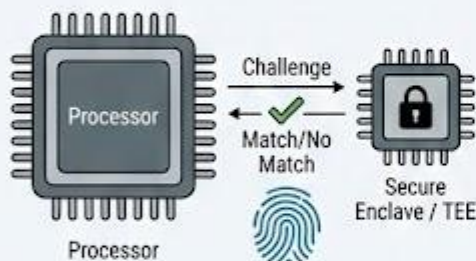


Each app is a unique user. Kernel prevents access.

Slide 4: iOS Security Model (The Walled Garden)



Slide 5: Biometrics & The Secure Enclave



Slide 6: Standard Mapping – NIST SP 800-53

NIST

Access Control (AC)



AC-3: Access
Enforcement



AC-19: Mobile
Access Control

Slide 7: Standard Mapping – ISO/IEC 27001:2022



Annex A Controls

- 8.1** User Endpoint Devices (Policies)
- 5.15** Access Control (Rules)
- 8.2** Privileged Access Rights (Restrict Rooting)

Slide 8: Data Protection – Encryption



FDE

Full Disk Encryption
(One Key)



FBE

File-Based Encryption
(Unique Keys per File)

FBE allows Direct Boot & Granular Security.

Slide 9: Network Security – Data in Transit



Android: Network Security Config
iOS: App Transport Security (ATS)

Enforces TLS/VPN to prevent MitM attacks.

Slide 10: Summary & The Rooting/Jailbreaking Risk



Jailbreaking = Superuser Access
Breaks Sandboxing
Violates ISO 27001 Integrity
Enterprise MDM blocks rooted devices.

Excellent foundation. Below is an **expanded, “meaty” version of Module 2** for **all three courses**, written **at the level of undergraduate students** but with **exam-ready depth, practical examples, and audit / governance links**.

You can paste this directly into lecture notes or convert it into slides.

COURSE 1: CMT 451 – Mobile Operating Systems Security

Module 2: Android vs iOS Architecture, Sandboxing & System Integrity (Expanded)

Slide 1: Module Overview & Learning Objectives (Expanded)

Why this module matters

- Mobile phones are now **primary computing devices**, holding:
 - Banking apps
 - Corporate email
 - Authentication tokens (MFA)
- A compromised mobile OS = **total loss of confidentiality, integrity, and availability**

By the end of this module, students should be able to:

- Explain **how Android and iOS enforce isolation**
 - Describe **how trust is established at boot time**
 - Analyze **why rooting/jailbreaking is a critical enterprise risk**
 - Map mobile OS controls to **information security standards**
-

Slide 2: Android Security Architecture – Linux at the Core

Kernel-Level Security

- Android runs on a **modified Linux kernel**
- Linux provides:
 - Process isolation
 - Memory protection
 - File system permissions

The “App-as-a-User” Model (Critical Concept)

- Every installed app is assigned:
 - A **unique Linux UID**
 - A **private data directory**

- Example:
 - WhatsApp ≠ Chrome ≠ Banking App
 - Even if malware exists, it is **confined to its own UID**

Why this matters

- Malware cannot:
 - Read another app's memory
 - Access another app's files
 - This is **OS-enforced**, not app-enforced
-

Slide 3: iOS Security Architecture – The Walled Garden

Core OS

- Built on **Darwin**, derived from BSD Unix + Mach kernel

Mandatory Code Signing

- Every executable must be:
 - Signed by Apple
 - Verified before execution
- Unsigned code **will not run**, even with user consent

Exploit Mitigations

- **ASLR**: Randomizes memory locations
- **DEP/XN bits**: Prevent execution of injected code
- **Pointer Authentication Codes (PAC)**: Protects against control-flow attacks

Security Trade-off

- iOS sacrifices flexibility for **strong security guarantees**
-

Slide 4: System Integrity – Bootstrapping Trust

The Chain of Trust

1. **Hardware Root of Trust (ROM)**
2. Bootloader verification
3. Kernel verification
4. OS and system partition verification

Key Principle

Trust is not assumed – it is **verified at every stage**

Failure Scenarios

- Modified kernel
- Unsigned boot image
- Rootkit injection

Result

- Device:
 - Refuses to boot
 - Enters recovery / restricted mode
 - Triggers factory reset (some cases)
-

Slide 5: Biometric Security & Hardware Isolation

The Core Risk

- Biometric data **cannot be changed**
 - You can change a password
 - You cannot change fingerprints

Secure Design

- **Secure Enclave (iOS)**
- **Trusted Execution Environment – TEE (Android)**

How it Works

- Fingerprint sensor → Secure chip
- Main OS sends a challenge
- Secure chip replies **Yes / No**
- Biometric data **never leaves hardware**

Security Outcome

- Malware in Android/iOS **cannot steal biometrics**
 - Even with root, enclave remains isolated
-

Slide 6: Application Permissions & Privacy Enforcement

Android Permissions

- Declared in AndroidManifest.xml

- Two categories:
 - Normal (low risk)
 - Dangerous (camera, mic, location)

Runtime Permissions

- Introduced in Android 6+
- User grants permission **at use-time**
- Example:
 - “Allow Camera while using the app?”

iOS Privacy Model

- Permission prompts are:
 - Mandatory
 - Non-bypassable
 - App Tracking Transparency:
 - Explicit consent for cross-app tracking
-

Slide 7: Data-at-Rest Encryption (Deep Dive)

Full Disk Encryption (FDE)

- Single key decrypts entire disk
- Weakness:
 - All data exposed after unlock

File-Based Encryption (FBE)

- Each file has:
 - Unique encryption key
- Supports **Direct Boot**
 - Alarms, calls work before unlock

Security Benefit

- Limits data exposure
 - Supports least privilege at the file level
-

Slide 8: Network Security & OS-Level Controls

Primary Threat

- Man-in-the-Middle attacks on:
 - Public Wi-Fi
 - Rogue access points

Android Protections

- Network Security Configuration:
 - Apps cannot trust user-installed CAs by default
 - Blocks common interception tools

iOS Protections

- App Transport Security (ATS):
 - Enforces HTTPS
 - Strong TLS
 - Secure ciphers
-

Slide 9: Standard Mapping – ISO/IEC 27001

Relevant Controls

- **8.1 – User Endpoint Devices**
 - Screen locks, encryption, MDM
- **8.2 – Privileged Access Rights**
 - Root/jailbreak restrictions
- **5.15 – Access Control**
 - App permissions, least privilege

Audit Perspective

- Mobile OS security = **organizational risk**, not personal preference
-

Slide 10: Jailbreaking & Rooting – Why Enterprises Block It

What Happens When You Root

- Sandbox collapses
- UID isolation is bypassed
- Malware gains system-wide visibility

Real Impact

- Banking trojans read OTPs

- Spyware captures screen contents
- Enterprise credentials stolen

Enterprise Response

- MDM detects compromise
 - Device blocked from:
 - Email
 - VPN
 - Corporate apps
-
-

COURSE 2: CMT 446 – IT Risk Management & Internal Controls

Module 2: Risk Assessment & Control Frameworks (Expanded)

Slide 1: Module Overview – Why Risk Management Exists

Reality Check

- Zero risk does not exist
- Security is about **risk-informed decisions**

Key Idea

Risk management is a **business process**, not an IT checklist

Audit Role

- Validate:
 - Risks are identified
 - Controls are appropriate
 - Residual risk is acceptable
-

Slide 2: The Risk Equation (Fully Explained)

Risk Exists When:

- A valuable **asset**
- Is exposed to a **threat**
- Through a **vulnerability**

Example

- Asset: Customer database
- Threat: Hacker
- Vulnerability: Unpatched web server

No vulnerability = No risk, even if threat exists

Slide 3: Risk Management Lifecycle (In Practice)

1. Identify

- Asset register
- Threat catalog

2. Assess

- Likelihood × Impact

3. Treat

- Controls selected

4. Monitor

- Continuous reassessment

Why Monitoring Matters

- Threats evolve faster than policies
-

Slide 4: Quantitative Risk Assessment (Numbers Matter)

Formulas

- $SLE = \text{Asset Value} \times \text{Exposure Factor}$
- $ALE = SLE \times ARO$

Example

- Asset value: \$100,000
- Exposure: 30%
- ARO: 1 per year
- $ALE = \$30,000$

Decision Logic

- Spending \$10k on controls makes sense
 - Spending \$100k does not
-

Slide 5: Qualitative Risk Assessment

Why Use It

- Not everything has a price:
 - Reputation
 - Trust
 - Regulatory penalties

Tools

- Risk heat maps
- H/M/L scoring

Strength

- Strategic decision-making
 - Board-level communication
-

Slide 6: COSO Internal Control Framework

COSO

Focus

- Enterprise-wide risk management

Five Components

1. Control Environment
2. Risk Assessment
3. Control Activities
4. Information & Communication
5. Monitoring

Used By

- Boards
 - Auditors
 - Regulators
-

Slide 7: COBIT 2019 – IT Governance Lens

COBIT 2019

Purpose

- Align IT with business value

Key Question

Is IT enabling strategy or just consuming budget?

Audit Value

- Evaluates:
 - Effectiveness
 - Efficiency
 - Alignment
-

Slide 8: Types of Internal Controls (Expanded)

Type	Purpose	Example
Preventive	Stop incident	MFA
Detective	Detect incident	SIEM
Corrective	Recover	Backups
Directive	Guide behavior	Policies

Best Practice

- Layer all four types
-

Slide 9: Standard Mapping – ISO/IEC 27001

Key Controls

- 5.7 – Threat Intelligence
- 8.8 – Vulnerability Management
- 8.28 – Secure Development

Audit Insight

- Risk management must be **documented and repeatable**
-

Slide 10: Risk Treatment Strategies (Decision Logic)

- **Mitigate:** Reduce risk
- **Transfer:** Insurance, outsourcing
- **Accept:** Low impact / cost

- **Avoid:** Stop activity

Critical Exam Insight

Acceptance must be **approved**, not accidental

COURSE 1: CMT 451 – Mobile Operating Systems Security

Module 2: Android vs iOS Architecture, Sandboxing & System Integrity (Expanded)

Slide 1: Module Overview & Learning Objectives (Expanded)

Why this module matters

- Mobile phones are now **primary computing devices**, holding:
 - Banking apps
 - Corporate email
 - Authentication tokens (MFA)
- A compromised mobile OS = **total loss of confidentiality, integrity, and availability**

By the end of this module, students should be able to:

- Explain **how Android and iOS enforce isolation**
 - Describe **how trust is established at boot time**
 - Analyze **why rooting/jailbreaking is a critical enterprise risk**
 - Map mobile OS controls to **information security standards**
-

Slide 2: Android Security Architecture – Linux at the Core

Kernel-Level Security

- Android runs on a **modified Linux kernel**
- Linux provides:
 - Process isolation
 - Memory protection
 - File system permissions

The “App-as-a-User” Model (Critical Concept)

- Every installed app is assigned:
 - A **unique Linux UID**
 - A **private data directory**
- Example:

- WhatsApp ≠ Chrome ≠ Banking App
- Even if malware exists, it is **confined to its own UID**

Why this matters

- Malware cannot:
 - Read another app's memory
 - Access another app's files
 - This is **OS-enforced**, not app-enforced
-

Slide 3: iOS Security Architecture – The Walled Garden

Core OS

- Built on **Darwin**, derived from BSD Unix + Mach kernel

Mandatory Code Signing

- Every executable must be:
 - Signed by Apple
 - Verified before execution
- Unsigned code **will not run**, even with user consent

Exploit Mitigations

- **ASLR**: Randomizes memory locations
- **DEP/XN bits**: Prevent execution of injected code
- **Pointer Authentication Codes (PAC)**: Protects against control-flow attacks

Security Trade-off

- iOS sacrifices flexibility for **strong security guarantees**
-

Slide 4: System Integrity – Bootstrapping Trust

The Chain of Trust

1. **Hardware Root of Trust (ROM)**
2. Bootloader verification
3. Kernel verification
4. OS and system partition verification

Key Principle

Trust is not assumed – it is **verified at every stage**

Failure Scenarios

- Modified kernel
- Unsigned boot image
- Rootkit injection

Result

- Device:
 - Refuses to boot
 - Enters recovery / restricted mode
 - Triggers factory reset (some cases)
-

Slide 5: Biometric Security & Hardware Isolation

The Core Risk

- Biometric data **cannot be changed**
 - You can change a password
 - You cannot change fingerprints

Secure Design

- **Secure Enclave (iOS)**
- **Trusted Execution Environment – TEE (Android)**

How it Works

- Fingerprint sensor → Secure chip
- Main OS sends a challenge
- Secure chip replies **Yes / No**
- Biometric data **never leaves hardware**

Security Outcome

- Malware in Android/iOS **cannot steal biometrics**
 - Even with root, enclave remains isolated
-

Slide 6: Application Permissions & Privacy Enforcement

Android Permissions

- Declared in AndroidManifest.xml
- Two categories:

- Normal (low risk)
- Dangerous (camera, mic, location)

Runtime Permissions

- Introduced in Android 6+
- User grants permission **at use-time**
- Example:
 - “Allow Camera while using the app?”

iOS Privacy Model

- Permission prompts are:
 - Mandatory
 - Non-bypassable
 - App Tracking Transparency:
 - Explicit consent for cross-app tracking
-

Slide 7: Data-at-Rest Encryption (Deep Dive)

Full Disk Encryption (FDE)

- Single key decrypts entire disk
- Weakness:
 - All data exposed after unlock

File-Based Encryption (FBE)

- Each file has:
 - Unique encryption key
- Supports **Direct Boot**
 - Alarms, calls work before unlock

Security Benefit

- Limits data exposure
 - Supports least privilege at the file level
-

Slide 8: Network Security & OS-Level Controls

Primary Threat

- Man-in-the-Middle attacks on:

- Public Wi-Fi
- Rogue access points

Android Protections

- Network Security Configuration:
 - Apps cannot trust user-installed CAs by default
 - Blocks common interception tools

iOS Protections

- App Transport Security (ATS):
 - Enforces HTTPS
 - Strong TLS
 - Secure ciphers
-

Slide 9: Standard Mapping – ISO/IEC 27001

Relevant Controls

- **8.1 – User Endpoint Devices**
 - Screen locks, encryption, MDM
- **8.2 – Privileged Access Rights**
 - Root/jailbreak restrictions
- **5.15 – Access Control**
 - App permissions, least privilege

Audit Perspective

- Mobile OS security = **organizational risk**, not personal preference
-

Slide 10: Jailbreaking & Rooting – Why Enterprises Block It

What Happens When You Root

- Sandbox collapses
- UID isolation is bypassed
- Malware gains system-wide visibility

Real Impact

- Banking trojans read OTPs
- Spyware captures screen contents

- Enterprise credentials stolen

Enterprise Response

- MDM detects compromise
- Device blocked from:
 - Email
 - VPN
 - Corporate apps

CMT 451 – Mobile Operating Systems Security

Module 2 Quiz

Questions

Q1. What is the primary role of the mobile operating system in security?

- A. Providing a user interface
- B. Acting as the trusted computing base
- C. Managing app updates
- D. Improving battery performance

Q2. In Android's security model, each application is:

- A. Executed under the same system user
- B. Assigned a shared memory space
- C. Assigned a unique Linux User ID (UID)
- D. Isolated only at the application layer

Q3. What security mechanism prevents unsigned apps from running on iOS?

- A. App sandboxing
- B. Mandatory code signing
- C. Runtime permissions
- D. Secure boot only

Q4. The "chain of trust" in mobile devices starts from:

- A. The operating system
- B. The application layer
- C. Hardware-based root of trust
- D. The mobile network

Q5. Why are biometric credentials stored in a Secure Enclave or TEE?

- A. To improve authentication speed
- B. To reduce storage costs
- C. To isolate sensitive data from the main OS
- D. To allow cloud synchronization

Q6. Which Android feature allows some functions (like alarms) before device unlock?

- A. Full Disk Encryption
- B. File-Based Encryption with Direct Boot
- C. Secure Boot
- D. App Sandboxing

Q7. What is the main risk of rooting or jailbreaking a mobile device?

- A. Reduced battery life
- B. Loss of app updates
- C. Breakdown of application sandboxing
- D. Loss of biometric authentication

Q8. App Transport Security (ATS) in iOS primarily enforces:

- A. VPN usage
- B. Application sandboxing
- C. Secure HTTPS communication
- D. Firewall filtering

Q9. Which Android file declares required application permissions?

- A. build.gradle
- B. permissions.xml
- C. AndroidManifest.xml
- D. app.config

Q10. Why do enterprises block rooted devices using MDM solutions?

- A. Rooted devices are slower
- B. Rooting violates app store rules
- C. Rooting bypasses OS security controls
- D. Rooted devices cannot encrypt data

Answers

1. **B**
2. **C**
3. **B**
4. **C**
5. **C**
6. **B**
7. **C**
8. **C**
9. **C**
10. **C**